A Formal Specification of the EvolutionaryGenerator v2.1
0/1 Knapsack Instance Generator

Mathematical model, control flow and as-implemented pseudocode
for the Java reference and the Python translation

Paola Azeneth Castillo Gutiérrez
Tecnológico de Monterrey
Original Java artefact: José Carlos Ortiz Bayliss

Abstract

This document specifies, in formal terms, the evolutionary generator of 0/1 Knapsack Problem (0/1-KP) instances distributed as `EvolutionaryGenerator v2.1`, in both its original Java form and its Python translation. The specification is descriptive rather than normative: it states what the code does, not what it was intended to do. Every claim reported here was obtained by direct inspection of the source together with executable verification (the Java sources were compiled and run, and the Python package was executed, within the same working session). Section 3 formalises the genotype-to-phenotype map and characterises exactly the region of instance space the generator can reach. Section 4 formalises the sixteen implemented fitness functions and the two declared but unreachable ones. Section 5 formalises the evolutionary engine, including an exact accounting of the evaluation budget. Section 6 describes the random number architecture, which is a tree of coupled streams rather than a single stream. Section 8 reports the measured equivalence and the measured cost ratio between the two implementations. Section 9 collects the points at which the implementation and its documented interface disagree.

Contents

1 Scope, sources and method of verification

1.1 Artefacts under study

Two artefacts are specified. Both live in the repository `MCCThesisAD2026`:

Artefact	Directory	Declared source level
Java reference	`EvolutionaryGenerator_v2.1_Java`	See Table 1
Python translation	`EvolutionaryGenerator_v2.1_Python`	Python $\geq$ 3.9, no external dependencies

The Java artefact is organised as three NetBeans projects, which declare mutually inconsistent source and target levels.

Table 1: Declared `javac` source and target levels per NetBeans project, read from each `nbproject/project.properties`.

Project	Java package root	`javac.source`	`javac.target`
`Generator`	`mx.tec.hermes`	11	11
`MetaH`	`mx.tec.meta`	1.8	1.8
`Utils`	`Utils`	1.7	1.7

Remark 1. The three declarations are not contradictory at the language level (the union compiles under the highest of the three), but they are evidence that the build descriptors were never resynchronised after the modules were revised at different times. The effective floor is Java 11.

1.2 Method of verification

Three levels of evidence are used, and the text distinguishes between them.

(V1) **Source inspection.** A statement about control flow or about which identifier is read where. These are checkable by reading the file cited.

(V2) **Executable verification.** A statement confirmed by compiling and running the code. The Java sources were compiled with `javac` 21 and the reference harness was executed; the Python package was executed under CPython.

(V3) **Measurement.** A numeric quantity obtained by timing or counting a live run. Measurements are single-machine and are reported as orders of magnitude, not as portable constants.

Remark 2 (Assumption stated explicitly)**.** Timings in Section 8 were taken on a single containerised core with no attempt at statistical control (no repetitions beyond a warm second run, no isolation of the Java JIT warm-up). They support a claim about the *ratio* between the two implementations and about the *order of magnitude* of the per-instance cost. They do not support a claim about absolute throughput on any other machine.

2 Preliminaries: the problem being generated

Definition 1 (0/1 Knapsack instance)**.** A 0/1-KP instance is a triple $I = (n, c, \{(w_i, p_i)\}_{i=1}^{n})$ with $n \in \mathbb{N}$ items, capacity $c \in \mathbb{N}$, integer weights $w_i \in \mathbb{N}$ and profits $p_i \in \mathbb{R}_{>0}$. A solution is a vector $x \in \{0,1\}^n$, and the problem is

$$\max_{x \in \{0,1\}^n} \sum_{i=1}^{n} p_i x_i \quad \text{subject to} \quad \sum_{i=1}^{n} w_i x_i \leq c.$$

Definition 2 (Instance space)**.** $\mathcal{I}$ denotes the set of all such triples. The generator does not search $\mathcal{I}$; it searches a strict subset determined by its encoding, characterised in Proposition 2.

2.1 The heuristic portfolio

The generator evaluates every candidate instance with a fixed portfolio of four deterministic constructive heuristics,

$$\mathcal{H} = \{\text{DEF},\ \text{MAXP},\ \text{MAXPW},\ \text{MINW}\}.$$

Each heuristic is defined by a priority relation $\prec_h$ on item indices, with ties broken by increasing index:

Heuristic	String in code	Priority key (best first)
DEF	`"DEFAULT"`	index i, increasing
MAXP	`"MAX_PROFIT"`	p_i, decreasing
MAXPW	`"MAX_PROFIT/WEIGHT"`	p_i/w_i, decreasing
MINW	`"MIN_WEIGHT"`	w_i, increasing

Definition 3 (Constructive solution and objective value)**.** Let $\sigma_h(I) = (i_1, i_2, \ldots, i_n)$ be the permutation of item indices induced by $\prec_h$. The packed set is built by the single pass

$$S_h(I) = \Big\{\, i_k \ :\ w_{i_k} \leq c - \textstyle\sum_{j<k,\ i_j \in S_h(I)} w_{i_j} \,\Big\},$$

and the objective value reached by heuristic h on instance I is

$$v_h(I) \ = \sum_{i \in S_h(I)} p_i.$$

The implementation does not perform this single sorted pass. It repeatedly scans the surviving item list and selects the feasible argument extremum (`KP.nextItem`), removing the chosen item from the list. The two formulations agree, which matters because the standard definition is the one used in the literature while the implemented one is the one that determines running time.

Proposition 1 (Rescan equivalence)**.** *For every $h \in \mathcal{H}$ and every instance I, the repeated-argument-extremum loop implemented in* `KP.solve` *returns the same packed set $S_h(I)$ as the single sorted pass, provided ties are broken by first occurrence in list order in both.*

Proof. Residual capacity is non-increasing along the loop, because `pack` only decrements it and no item is ever removed from the knapsack. Feasibility of a fixed item i is the predicate $w_i \le c_{\text{res}}$, which is therefore monotone: once false it stays false. Hence the subsequence of items that are ever feasible is processed in $\prec_h$ order in both formulations, and both select exactly the feasible prefix of that subsequence. The strict comparison operators used in `nextItem` ($>$ for maximising keys, $<$ for minimising keys) retain the first occurrence among equal keys, which coincides with a stable sort. □

Corollary 1 (Complexity of the implemented heuristics)**.** *Each call to* `KP.solve` *runs in $\Theta(n^2)$ time in the worst case, since up to n selections are made and each performs a full linear scan of the surviving items. The sorted formulation of the same function runs in $O(n \log n)$. The two agree on output and disagree on cost by a factor $\Theta(n/\log n)$.*

Remark 3 (Verification, level V2)**.** Proposition 1 was checked empirically on 9,000 randomly generated instances ($3 \le n \le 15$, $5 \le c \le 60$, integer weights in $[1, 32]$ and profits in $[1, 64]$, three heuristics with a non-trivial key) with zero mismatches against a sort-once reference implementation.

3 The genotype-to-phenotype map

This is the component that determines which instances the generator can produce at all, and it is where the effective and the nominal parameter spaces separate.

3.1 Chromosome geometry

Let $W_{\max}, P_{\max} \in \mathbb{N}_{\ge 2}$ be the parameters named `maxWeight` and `maxProfit` in the public interface, and let n be the number of items. The code computes the bit budget as

$$b_w \;=\; \left\lceil \frac{\log_{10} W_{\max}}{\log_{10} 2} \right\rceil, \qquad b_p \;=\; \left\lceil \frac{\log_{10} P_{\max}}{\log_{10} 2} \right\rceil,$$

which is the change-of-base expression for $\lceil \log_2 \cdot \rceil$.

Remark 4 (Floating-point safety of the change of base)**.** The expression $\lceil \log_{10}(x)/\log_{10}(2) \rceil$ is not *a priori* equal to $\lceil \log_2 x \rceil$ in IEEE 754 double arithmetic, since the quotient may land just above an integer at a power of two. This was checked exhaustively for every integer $x \in [2, 10^5]$ and no discrepancy occurs, so we may write $b_w = \lceil \log_2 W_{\max} \rceil$ and $b_p = \lceil \log_2 P_{\max} \rceil$ without qualification in the range of practical interest.

Define the per-item block length and the chromosome length

$$\ell \;=\; b_w + b_p, \qquad L \;=\; n\,\ell, \qquad \Omega \;=\; \{0,1\}^L.$$

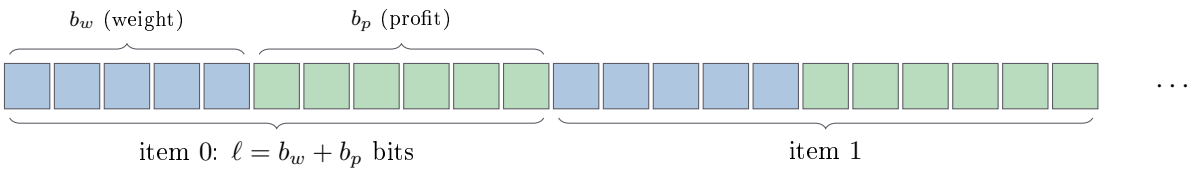

Figure 1: Chromosome layout for the default configuration ($W_{\max} = 20 \Rightarrow b_w = 5$; $P_{\max} = 50 \Rightarrow b_p = 6$; $\ell = 11$; $n = 10 \Rightarrow L = 110$). Within a block, bit offset j carries weight 2^j (little-endian).

3.2 The decoding map

Definition 4 (Decoding). The decoding map $\Gamma_\theta : \Omega \to \mathcal{I}$, parameterised by $\theta = (n, c, W_{\max}, P_{\max})$, sends $x \in \Omega$ to the instance with capacity c and, for each $i \in \{0, \dots, n-1\}$,

$$w_i(x) \;=\; 1 + \sum_{j=0}^{b_w-1} x_{\,i\ell+j}\, 2^j, \qquad p_i(x) \;=\; 1 + \sum_{j=0}^{b_p-1} x_{\,i\ell+b_w+j}\, 2^j.$$

The additive constant 1 is the `+1` at the end of `KPIndividual.toInteger` and exists to exclude zero weights and zero profits.

Proposition 2 (Exact reachable set). *For fixed θ, the decoding map is a bijection*

$$\Gamma_\theta : \{0,1\}^L \;\longrightarrow\; \Big(\{1, \dots, 2^{b_w}\} \times \{1, \dots, 2^{b_p}\}\Big)^n,$$

and consequently the set of instances the generator can emit is exactly

$$\mathcal{I}_\theta = \Big\{ (n, c, \{(w_i, p_i)\}) \;:\; w_i \in [1, 2^{\lceil \log_2 W_{\max} \rceil}],\; p_i \in [1, 2^{\lceil \log_2 P_{\max} \rceil}] \Big\}, \qquad |\mathcal{I}_\theta| = 2^L.$$

Proof. Each block of b_w bits ranges over $\{0, \dots, 2^{b_w} - 1\}$ bijectively under the little-endian value map, and adding 1 is a bijection onto $\{1, \dots, 2^{b_w}\}$; identically for profits. Blocks are disjoint and the product of bijections is a bijection. The capacity c is a constant of θ and is not encoded. □

Two consequences follow, both of which bear directly on any claim about the coverage of the instance space.

Corollary 2 (The declared bounds are not enforced). *Unless $W_{\max}$ is a power of two, $\max_i w_i$ may exceed $W_{\max}$; identically for $P_{\max}$. The attained supremum is $2^{\lceil \log_2 W_{\max} \rceil}$, so the relative overshoot is bounded by*

$$\frac{2^{\lceil \log_2 W_{\max} \rceil}}{W_{\max}} \;<\; 2.$$

For the default configuration ($W_{\max} = 20$, $P_{\max} = 50$) the attainable ranges are $w_i \in [1, 32]$ and $p_i \in [1, 64]$.

Remark 5 (Verification, level V2). A single decoded individual under the default configuration produced

$$w = (31,\ 14,\ 7,\ 21,\ 27,\ 1,\ 2,\ 13,\ 12,\ 19), \qquad p = (1,\ 23,\ 58,\ 26,\ 34,\ 11,\ 15,\ 28,\ 64,\ 15).$$

The values $31 > 20$ and $64 > 50$ exhibit Corollary 2 directly.

Proposition 3 (Dyadic collapse of the parameter space). *The pair $(W_{\max}, P_{\max})$ influences the generator only through $(b_w, b_p) = (\lceil \log_2 W_{\max} \rceil, \lceil \log_2 P_{\max} \rceil)$. The map $W_{\max} \mapsto b_w$ is constant on each dyadic interval $(2^{k-1}, 2^k]$. Hence for a nominal grid $W_{\max}, P_{\max} \in \{2, \dots, N\}$, the number of behaviourally distinct configurations is*

$$\big|\{(b_w, b_p)\}\big| \;=\; \lceil \log_2 N \rceil^2,$$

against a nominal cardinality of $(N-1)^2$.

Proof. $W_{\max}$ and $P_{\max}$ appear in the source only inside the four $\lceil \log_{10}(\cdot) / \log_{10}(2) \rceil$ expressions of `KPIndividual` (the field initialiser, the three static setters, `toKP` and `toString`) and inside the output file name assembled by `KP.generate`. No other read exists. The interval statement is the definition of the ceiling of a logarithm. □

Table 2: Collapse of the nominal $(W_{\max}, P_{\max})$ grid onto behaviourally distinct configurations. Computed exhaustively.

Grid	Nominal pairs	Distinct (b_w, b_p)	Collapse ratio
$[2, 100]^2$	9,801	49	200:1
$[2, 200]^2$	39,601	64	619:1
$[2, 500]^2$	249,001	81	3,074:1
$[2, 1000]^2$	998,001	100	9,980:1

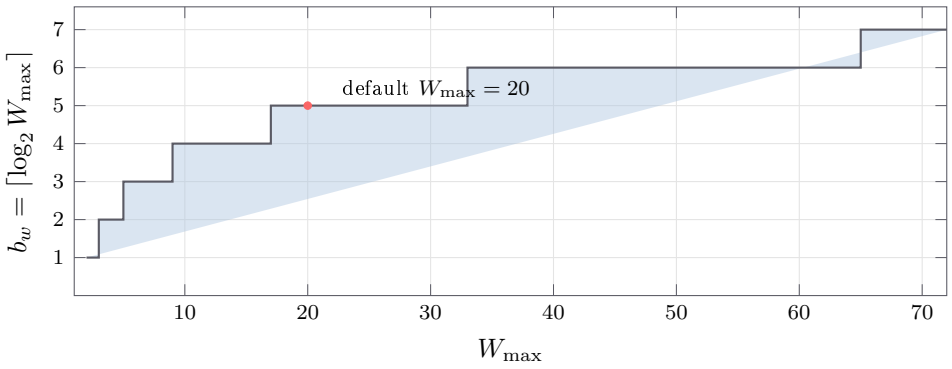

Figure 2: The effective parameter is a step function of the nominal one. Every $W_{\max} \in \{17, \ldots, 32\}$ yields an identical generator.

Remark 6 (Methodological consequence)**.** Proposition 3 is a statement about configurable instance generators in general and not about this artefact in particular: whenever a declared range parameter enters the system only through a bit-width computation, sweeping that parameter on a linear grid produces far fewer distinct experimental conditions than the grid suggests. Any published factorial design over $(W_{\max}, P_{\max})$ that assumes one condition per grid point overstates its own resolution by the ratio in Table 2.

4 The fitness landscape

4.1 Objective vector and the guarded extrema

For a genotype x write the objective vector

$$v(x) \;=\; \big(v_{\text{DEF}},\; v_{\text{MINW}},\; v_{\text{MAXPW}},\; v_{\text{MAXP}}\big)(\Gamma_\theta(x)) \;\in\; \mathbb{R}^4_{\geq 0}.$$

Each component requires an independent decoding, because `KP.solve` consumes the item list destructively; `KPEvaluator` therefore calls `toKP` four times per evaluation.

The code does not use the mathematical extrema but the guarded ones defined in `Statistical`:

$$\max^\dagger(A) \;=\; \max\big(A \cup \{\varepsilon\}\big), \qquad \min^\dagger(A) \;=\; \min\big(A \cup \{M\}\big),$$

with $\varepsilon =$ `Double.MIN_VALUE` $= 4.9 \times 10^{-324}$ (the smallest *positive* double, not the most negative one) and $M =$ `Double.MAX_VALUE`.

Remark 7 (When the guard is observable)**.** $\min^\dagger$ is inert for this application, since all objective values are bounded above by the total profit. $\max^\dagger$ is observable exactly when $v(x) = \mathbf{0}$, that is when no item fits the knapsack in any heuristic order, equivalently $\min_i w_i > c$. In that degenerate case $\max^\dagger$ returns ε rather than 0, and the EASY fitness values become $-\varepsilon$ rather than 0. The effect is numerically negligible but it breaks the sign symmetry of the fitness at the degenerate point.

4.2 The sixteen implemented objectives

Let $\lambda = 2$ and let $s(\cdot)$ denote the sample standard deviation with denominator $m - 1$.

Definition 5 (Fitness functions)**.** For a target heuristic $h \in \mathcal{H}$ write $\mathcal{H}_{-h} = \mathcal{H} \setminus \{h\}$ and $v_{-h}(x) = \{v_g(x) : g \in \mathcal{H}_{-h}\}$. The generator maximises $f_\tau : \Omega \to \mathbb{R}$, where

$$f_{h\text{-EASY}}(x) = v_h(x) - \max^\dagger \big(v_{-h}(x)\big), \tag{1}$$

$$f_{h\text{-HARD}}(x) = \min^\dagger \big(v_{-h}(x)\big) - v_h(x), \tag{2}$$

$$f_{\text{MAX-VAR}}(x) = s\big(v(x)\big), \tag{3}$$

$$f_{\text{MIN-VAR}}(x) = -\,s\big(v(x)\big). \tag{4}$$

For an unordered target pair $\{a, b\} \subset \mathcal{H}$ with complement $\{c, d\}$, let $\hat{v}_g(x) = v_g(x)/\max^\dagger(v(x))$ be the max-normalised objectives. Then

$$f_{\text{PAIRED}(a,b)}(x) = \min\big(\hat{v}_a, \hat{v}_b\big) - \max\big(\hat{v}_c, \hat{v}_d\big) - \lambda\,\big|\hat{v}_a - \hat{v}_b\big|. \tag{5}$$

Equations (1) and (2) instantiate for four target heuristics each (8 objectives); (3) and (4) give 2; (5) instantiates for the $\binom{4}{2} = 6$ unordered pairs. The total is 16.

Remark 8 (Interpretation)**.** (1) rewards a strict performance gap in favour of h over the best competitor, so its maximiser is an instance on which h is the uniquely correct choice. (2) is its mirror: it rewards h falling below the worst competitor. (5) rewards two heuristics jointly dominating the other two while penalising, with strength λ, any separation between the two members of the pair. The normalisation makes the pair objectives scale-free, which the four unpaired families are not.

4.3 The two unreachable declarations

Proposition 4 (Unreachable enumeration members)**.** *The enumeration* `KPEvaluator.InstanceType` *declares* 18 *members. Two of them,* `MARKOVITZ_EASY` *and* `MARKOVITZ_HARD`*, have no corresponding* `case` *label in the dispatch of* `evaluate`*. Any call carrying either value reaches the* `default` *branch, which writes a diagnostic to standard error and terminates the process with* `System.exit(1)`*. The effective type space therefore has cardinality* 16*, not* 18*.*

Remark 9. The Python translation reproduces this behaviour deliberately: its `if/elif` chain has no branch for either member, and the terminal `else` calls `sys.exit(1)`. This is a faithfulness decision, not an independent defect.

5 The evolutionary engine

5.1 State and operators

Definition 6 (Population state)**.** The engine maintains a population $\Pi_t = (\pi_t^1, \ldots, \pi_t^{\mu_t})$ of individuals, each a pair $\pi = (x, \phi)$ with $x \in \Omega$ and cached fitness $\phi = f_\tau(x)$, together with an external elite β. The population is kept sorted by ϕ in non-increasing order (the objective is MAXIMIZE throughout `KP.generate`).

Selection. `TournamentSelector` draws two parents independently. Each draw samples k individuals uniformly with replacement from Π_t, sorts them, and returns a *copy* of the best. Selection pressure is the standard tournament pressure: for a uniformly random individual of rank r (best $= 1$),

$$\Pr[\text{rank } r \text{ wins}] \;=\; \left(\frac{\mu - r + 1}{\mu}\right)^k - \left(\frac{\mu - r}{\mu}\right)^k.$$

Crossover. `KPIndividual.combine` draws $q \sim \mathcal{U}\{0, \ldots, L-1\}$ and exchanges the prefixes $x_{0:q}$ of the two parents in place, returning the two mutated parent objects themselves rather than new objects.

Proposition 5 (The crossover rate is inert)**.** *The parameter* `crossoverRate` *is accepted by* `KP.generate`*, forwarded to* `GeneticAlgorithm.run`*, clamped to* $[0, 1]$*, and passed to* `Individual.combine`*. The body of* `KPIndividual.combine` *never reads it. Consequently crossover is applied with probability* 1 *for every selected pair, for every value of the parameter.*

Remark 10 (Evidence that this is an omission rather than a design choice)**.** The sibling implementation `GPIndividual.combine`, in the same `MetaH` module and against the same abstract signature, guards its recombination with `if (random.nextDouble() < crossoverRate)`. The contract is therefore honoured by one subclass and silently dropped by the other.

Remark 11 (In-place mutation is safe here)**.** Returning the parent objects would corrupt the population if the parents were population members. They are not: `TournamentSelector` returns `tmp.get(0).copy()`. The aliasing is contained by that copy, so Proposition 5 is a defect of the parameter contract and not of population integrity.

Mutation. `KPIndividual.mutate` flips each of the L bits independently with probability p_m, so the number of flips is $\mathrm{Bin}(L, p_m)$ with mean Lp_m. For the default $L = 110$ and $p_m = 0.1$ this is 11 expected flips per individual per generation, which is a high mutation regime for a bit-string genetic algorithm.

Replacement. The engine is generational with an external elite. The whole population is replaced by the offspring; β is updated only on strict improvement, so the best-so-far trajectory is monotone even though the population trajectory is not.

5.2 Exact evaluation budget

Let μ be the requested population size and $E_{\max}$ the evaluation budget (hard-coded to 100,000 inside `KP.generate`). Define

$$\mu' \;=\; 2\left\lceil \frac{\mu}{2} \right\rceil \;=\; \begin{cases} \mu, & \mu \text{ even}, \\ \mu + 1, & \mu \text{ odd}. \end{cases}$$

Proposition 6 (Budget accounting and population drift)**.** *The initialisation consumes* μ *evaluations. The inner refill loop adds offspring two at a time while* $|\Pi_{\text{next}}| < |\Pi_t|$*, so the first generation produces* μ' *offspring and every later generation produces* μ'*. Therefore*

$$G \;=\; \left\lceil \frac{E_{\max} - \mu}{\mu'} \right\rceil, \qquad E_{\text{total}} \;=\; \mu + G\,\mu',$$

and the population size is μ *at* $t = 0$ *and* μ' *for all* $t \geq 1$*. When* μ *is odd the algorithm silently runs with a population one larger than requested and overshoots the evaluation budget.*

Remark 12 (Verification, level V2)**.** For $\mu = 10$, $E_{\max} = 1000$: predicted $G = 99$, $E_{\text{total}} = 1000$, final size 10; measured 1000 and 10. For $\mu = 11$, $E_{\max} = 1000$: predicted $\mu' = 12$, $G = 83$, $E_{\text{total}} = 1007$, final size 12; measured 1007 and 12.

Corollary 3 (Cost of one instance)**.** *Since each evaluation performs four calls to* `KP.solve`*, each* $\Theta(n^2)$ *by Corollary 1, the cost of generating one instance is*

$$\Theta\big(E_{\text{total}} \cdot 4n^2\big) \;=\; \Theta\big(E_{\max}\, n^2\big),$$

which for the hard-coded $E_{\max} = 100{,}000$ *is invariant in the population size and in every other exposed parameter except* n*.*

6 Randomness architecture

The generator does not draw from a single stream. It builds a tree of `java.util.Random` instances, and the leaves of that tree are attached to individuals rather than to the algorithm.

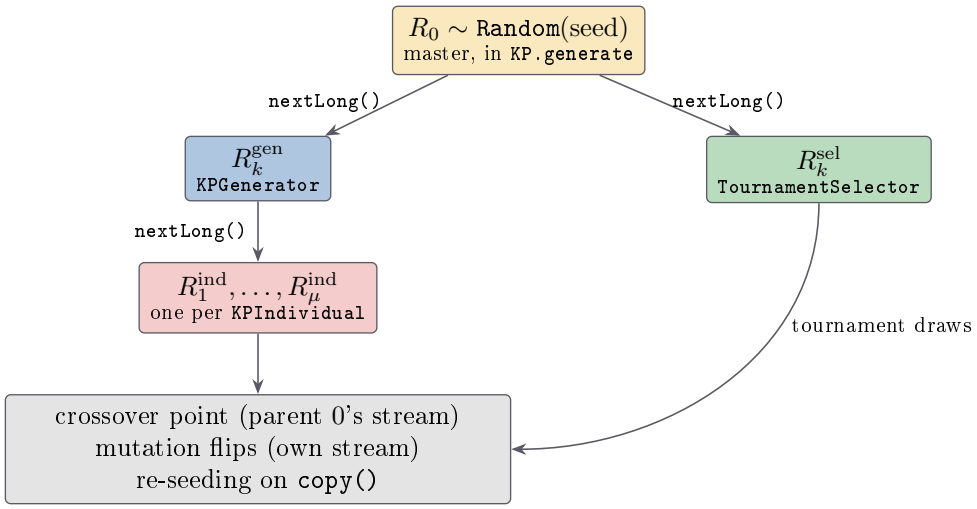

Figure 3: Seed tree for instance k. The master stream is consumed twice per instance, so instances within one call to `generate` are dependent but distinct.

Proposition 7 (`copy` is not free of side effects)**.** *The private copy constructor of* `KPIndividual` *seeds the new object with* `individual.random.nextLong()`*, that is, by drawing from the source object's own stream. Copying therefore advances the state of the copied individual. Since the tournament selector copies every tournament winner, and the genetic algorithm copies every new elite, the per-individual streams are advanced at a rate proportional to how often an individual wins a tournament. The leaf streams are consequently not independent of the selection process.*

Remark 13 (Verification, level V2)**.** Two individuals were constructed from the identical seed. In the first, a single `nextLong()` was drawn directly; in the second, `copy()` was invoked and then `nextLong()` was drawn. The two draws differ, which establishes that `copy()` consumed a draw from the source.

Remark 14 (Why this matters for reproducibility)**.** Reproducibility is preserved (the process is fully determined by the master seed), but *statistical* independence between the mutation streams of different individuals is not guaranteed, and any refactoring that changes how often `copy` is called will change the trajectory even at a fixed seed. This is a hard constraint on the parallelisation of the generator.

Proposition 8 (Shared mutable configuration blocks intra-process parallelism)**.** *The encoding parameters* `maxWeightPerItem`*,* `maxProfitPerItem`*,* `nbItems`*,* `capacity` *and the derived* `nbBits` *are declared* `static` *in* `KPIndividual` *and are written by* `KP.generate` *through static setters. Two concurrent generation runs with different θ inside one JVM therefore race on a shared encoding. The Python translation inherits the same structure as class attributes, so the same restriction applies to threads within one interpreter (though not to separate processes).*

7 Control flow, as implemented

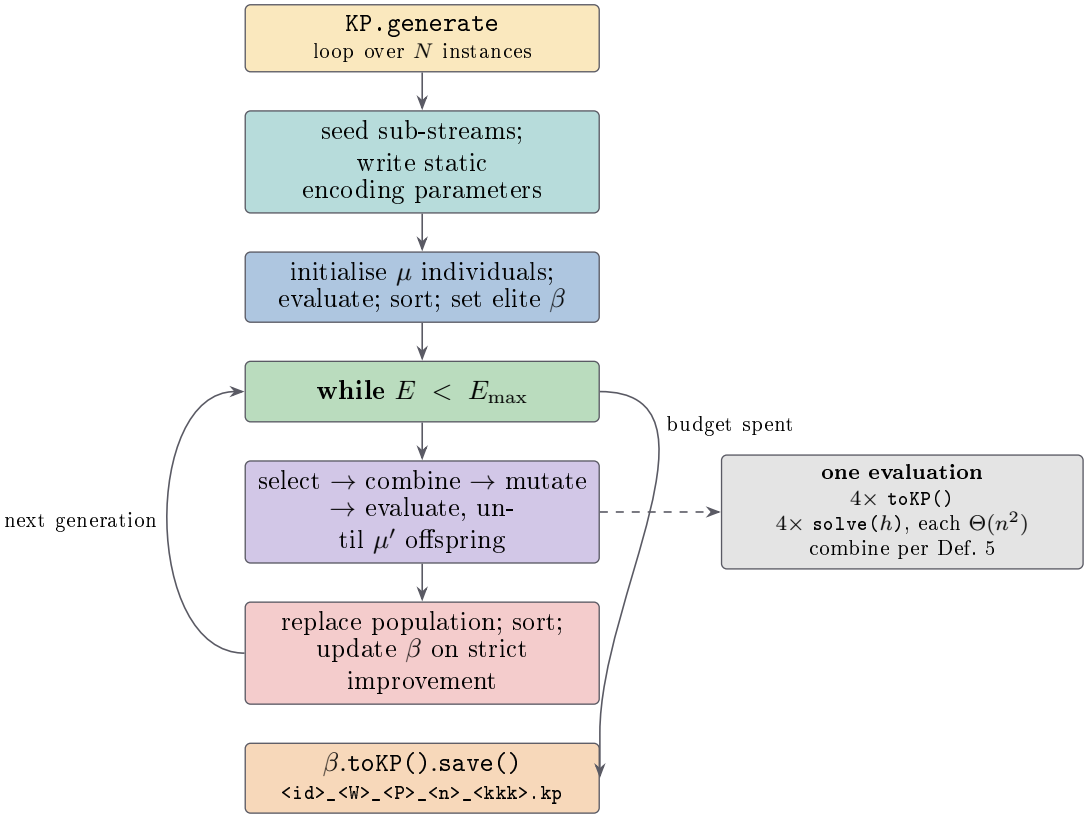

Figure 4: Control flow of one call to `KP.generate`. The structure is identical in both implementations.

7.1 Pseudocode

The following algorithms describe the code as it currently stands, including the behaviours identified above. Inert parameters are marked.

Algorithm 1 GENERATE (`KP.generate`), as implemented

Input: type τ, count N, id, path, n, c, $W_{\max}$, $P_{\max}$, μ, p_c *(inert)*, p_m, k, seed
Output: N files written to disk

$R_0 \leftarrow \textsc{Random}(\text{seed})$
for $j \leftarrow 0$ **to** $N-1$ **do**
  $\mathcal{E} \leftarrow \text{KP}\textsc{Evaluator}(\tau)$ ▷ evaluation counter resets per instance
  $\mathcal{G} \leftarrow \text{KP}\textsc{Generator}(R_0.\textsc{nextLong}())$
  $\mathcal{S} \leftarrow \textsc{TournamentSelector}(k,\ R_0.\textsc{nextLong}())$
  $\text{KP}\textsc{Individual}.\{c, W_{\max}, P_{\max}, n\} \leftarrow (c, W_{\max}, P_{\max}, n)$ ▷ static write; see Prop. 8
  $\beta \leftarrow \textsc{RunGA}(\mathcal{E}, \mathcal{G}, \mathcal{S}, \mu,\ 100000,\ p_c,\ p_m,\ \textsc{generational})$ ▷ budget hard-coded
  $\textsc{Save}(\beta.\text{toKP}(),\ \text{path} \| \text{id} \| W_{\max} \| P_{\max} \| n \| \textsc{Fmt}_{000}(j) \| \texttt{".kp"})$
end for

Algorithm 2 RunGA (`GeneticAlgorithm.run` + `runGenerational`)

Input: evaluator $\mathcal{E}$, generator $\mathcal{G}$, selector $\mathcal{S}$, μ, $E_{\max}$, p_c, p_m
Output: best individual found

if $\mu < 2$ **then**
  halt
end if
$\Pi \leftarrow (\mathcal{G}.\textsc{generate}() : i = 1..\mu)$
$p_c \leftarrow \text{clamp}(p_c, 0, 1)$; $p_m \leftarrow \text{clamp}(p_m, 0, 1)$ ▷ p_c clamped but never read
for all $\pi \in \Pi$ **do**
  $\pi.\phi \leftarrow \mathcal{E}.\textsc{evaluate}(\pi)$
end for
SortDescending(Π); $\beta \leftarrow \Pi[1].\textsc{copy}()$
while $\mathcal{E}.E < E_{\max}$ **do**
  $\Pi' \leftarrow \langle\,\rangle$
  while $|\Pi'| < |\Pi|$ **do**
    $(\pi_a, \pi_b) \leftarrow \mathcal{S}.\textsc{select}(\Pi)$ ▷ returns copies
    $(\pi_a, \pi_b) \leftarrow \pi_a.\textsc{combine}(\pi_b, p_c)$ ▷ same objects, mutated in place
    for all $\pi \in \{\pi_a, \pi_b\}$ **do**
      $\pi.\textsc{mutate}(p_m)$
      $\pi.\phi \leftarrow \mathcal{E}.\textsc{evaluate}(\pi)$
      append π to Π' ▷ overshoots by one when $|\Pi|$ is odd
    end for
  end while
  $\Pi \leftarrow \Pi'$; SortDescending(Π)
  if $\Pi[1].\phi > \beta.\phi$ **then**
    $\beta \leftarrow \Pi[1].\textsc{copy}()$
  end if
end while
return β

Algorithm 3 Select (`TournamentSelector.select`)

Input: population Π, tournament size k
Output: two individuals

for $i \leftarrow 1$ **to** 2 **do**
  $T \leftarrow \langle\,\rangle$
  for $j \leftarrow 1$ **to** k **do**
    append $\Pi[\,R^{\text{sel}}.\textsc{nextInt}(|\Pi|) + 1\,]$ to T ▷ uniform, with replacement
  end for
  SortDescending(T)
  $\pi_i \leftarrow T[1].\textsc{copy}()$ ▷ advances $T[1]$'s stream; Prop. 7
end for
return (π_1, π_2)

Algorithm 4 Genetic operators of `KPIndividual`

Constructor KPIndividual(seed)
$R^{\text{ind}} \leftarrow \text{Random}(\text{seed})$; $\phi \leftarrow 0$
for $i \leftarrow 0$ **to** $L-1$ **do**
    $x_i \leftarrow R^{\text{ind}}.\text{nextBoolean}()$
end for

Copy constructor KPIndividual(π)
$R^{\text{ind}} \leftarrow \text{Random}\big(\pi.R^{\text{ind}}.\text{nextLong}()\big)$ ▷ side effect on π
$\phi \leftarrow \pi.\phi$; $x \leftarrow \text{clone}(\pi.x)$

Combine combine(π, p_c)
$q \leftarrow R^{\text{ind}}.\text{nextInt}(L)$ ▷ p_c is not read
swap $x_{0:q}$ with $\pi.x_{0:q}$ **in place**
return (self, π)

Mutate mutate(p_m)
for $i \leftarrow 0$ **to** $L-1$ **do**
    if $R^{\text{ind}}.\text{nextDouble}() < p_m$ **then**
        flip x_i
    end if
end for

Decode toKP()
for $i \leftarrow 0$ **to** $n-1$ **do**
    $w_i \leftarrow 1 + \text{Value}(x_{\,i\ell\,:\,i\ell+b_w})$
    $p_i \leftarrow 1 + \text{Value}(x_{\,i\ell+b_w\,:\,(i+1)\ell})$
end for
return KP($\{(w_i, p_i)\},\ c$)

Algorithm 5 EVALUATE (`KPEvaluator.evaluate`)

Input: individual π, type τ
Output: fitness value

$E \leftarrow E + 1$
for all $h \in (\text{DEF}, \text{MINW}, \text{MAXPW}, \text{MAXP})$ **do**
  $I_h \leftarrow \pi.\text{TOKP}()$ ▷ fresh decode; SOLVE is destructive
  $\text{SOLVE}(I_h, h)$; $v_h \leftarrow I_h.\text{OBJVALUE}()$
end for
if $\tau \in \{h\text{-EASY}\}$ **then**
  return $v_h - \max^{\dagger}(v_{-h})$
else if $\tau \in \{h\text{-HARD}\}$ **then**
  return $\min^{\dagger}(v_{-h}) - v_h$
else if $\tau = \text{MAX-VAR}$ **then**
  return $s(v)$
else if $\tau = \text{MIN-VAR}$ **then**
  return $-s(v)$
else if $\tau = \text{PAIRED}(a, b)$ **then**
  $\hat{v} \leftarrow v / \max^{\dagger}(v)$
  return $\min(\hat{v}_a, \hat{v}_b) - \max(\hat{v}_c, \hat{v}_d) - \lambda|\hat{v}_a - \hat{v}_b|$
else
  halt ▷ reached by MARKOVITZ{EASY,HARD}; Prop. 4
end if

Algorithm 6 SOLVE (`KP.solve`), the constructive heuristic

Input: instance I with item list $\mathcal{L}$, heuristic h
Output: knapsack filled; $I.\mathcal{L}$ emptied of packed items

item $\leftarrow \text{NEXTITEM}(\mathcal{L}, h)$
while item $\neq \varnothing$ **do**
  PACK(item); remove item from $\mathcal{L}$
  item $\leftarrow \text{NEXTITEM}(\mathcal{L}, h)$ ▷ full rescan: $\Theta(n)$ per selection
end while

NEXTITEM$(\mathcal{L}, h)$
return the first (for DEF) or key-extremal (otherwise) item $\in \mathcal{L}$ with $w_{\text{item}} \le c_{\text{res}}$, or $\varnothing$

7.2 Output format

`KP.save` writes a plain text file: the first line carries the item count and the capacity, and each subsequent line carries a weight and a profit formatted with the pattern `0.000`. Profits are integers by Definition 4, so the three decimal places are always zero. The file name is assembled as

$$\texttt{<id>\_<}W_{\max}\texttt{>\_<}P_{\max}\texttt{>\_<}n\texttt{>\_<}jjj\texttt{>.kp},$$

which records the *nominal* parameters and not the effective ones (b_w, b_p). Two runs that differ in file name may therefore be behaviourally identical, by Proposition 3.

8 The Python implementation

8.1 What was translated and what was rebuilt

The Python package mirrors the three Java projects one-to-one (`hermes/`, `metah/`, `utils/`) and adds a compatibility layer, `javacompat/`, that has no Java counterpart because it reimplements pieces of the Java runtime.

Table 3: Module correspondence.

Java project	Python package	Role
`Generator`	`hermes/`	problem, heuristics, encoding, evaluator
`MetaH`	`metah/`	individuals, selectors, metaheuristics
`Utils`	`utils/`	statistical and file helpers
(none)	`javacompat/`	`Random`, `BitSet`, `DecimalFormat`, `Collections`
(none)	`main.py`	command line entry point

Remark 15 (Why `javacompat` is necessary and not merely convenient)**.** Bit-exact equivalence is impossible without reimplementing `java.util.Random`. Java specifies a 48-bit linear congruential generator with fixed multiplier and addend; CPython's `random` module uses the Mersenne Twister. Since the generator's entire trajectory is a deterministic function of that stream, substituting the Python generator would change every instance produced. The same argument applies, more weakly, to `BitSet` slicing semantics (where `get(from, to)` re-bases bit `from` to index 0) and to `DecimalFormat` rounding (HALF_EVEN over the exact decimal expansion of the double).

8.2 Measured equivalence

Proposition 9 (Bit-exact behavioural equivalence)**.** *Under an identical seed and configuration, the Python translation reproduces the output of the Java reference exactly, where "exactly" means that every floating point quantity agrees in its raw IEEE 754 bit pattern.*

This was verified end to end within the present session, at evidence level V2, by the following chain:

(1) The 28 Java source files were compiled from scratch with `javac` 21, together with the harness `tests/java/Reference.java`.

(2) The harness was executed live, producing 405 lines covering the statistical helpers, the encoding and genetic operators, the four heuristics, `solveCA`, `solveX`, dynamic programming, problem sets and streams, both genetic algorithm types, the micro genetic algorithm, the evolutionary strategy, and genetic programming.

(3) The live Java output was compared byte for byte against the stored golden file (`tests/expected_java_output.txt`). They are identical, which confirms that the golden file is a faithful record and not a stale one.

(4) `tests/test_equivalence.py` was executed and reproduced all 405 lines from the Python side.

Remark 16 (Scope of the claim)**.** Proposition 9 is a statement about a fixed scenario of substantial coverage, not a proof of equivalence over all inputs. It is strong evidence, not a formal refinement proof. In particular it does not exercise every one of the 16 fitness types.

8.3 Measured cost

Table 4: Cost of one full instance ($E_{\max} = 100{,}000$, $\mu = 10$, $n = 10$, $c = 20$, $W_{\max} = 20$, $P_{\max} = 50$, $k = 3$, $p_m = 0.1$, type MAXP-EASY). Single containerised core; evidence level V3.

Implementation	Evaluations	Time per instance	Time per evaluation
Java (OpenJDK 21)	100,000	1.44 s	14.4 μs
Python (CPython)	100,000	23.46 s	234.6 μs
Ratio			$\approx 16.3\times$

Corollary 4 (Experimental budget)**.** *On a single core, a campaign of M instances costs approximately $1.4M$ seconds in Java and $23.5M$ seconds in Python. For $M = 5{,}000$ this is about 2.0 hours and 32.6 hours respectively. Since instances within one call to* `generate` *are independent given their sub-seeds, the workload is embarrassingly parallel at the instance level, subject to the constraint of Proposition 8.*

Remark 17 (The refactoring argument, stated precisely)**.** Corollary 4 together with Proposition 8 gives the causal chain that justifies treating refactoring as a methodological precondition rather than as a contribution: an empirical characterisation of coverage requires a large sample; a large sample requires parallel execution; parallel execution within one process is blocked by the shared mutable encoding state in `KPIndividual`; therefore removing that shared state is a prerequisite for the analysis and not an end in itself.

8.4 Preserved anomalies

The translation deliberately reproduces behaviours of the original that a clean-room reimplementation would not have written. They are listed here because they are properties of the specified system, not accidents of the translation.

Table 5: Behaviours preserved by the Python translation for the sake of equivalence.

Site	Behaviour
`Statistical.max`	Accumulator initialised to the smallest positive double, so an all-negative input returns 4.9×10^{-324}.
`KPIndividual.combine`	Crossover rate ignored; parents mutated in place (Prop. 5).
`KPIndividual.toString`	Reads `bits.get(j + from)` on an already re-based slice, so the profit field always prints as zeros while the parenthesised value is correct.
`Knapsack.copy`	Constructs the copy with the *residual* capacity and then repacks, decrementing it a second time.
`KP.solve()` (DP)	Truncates to `int` inside a `double` table.
`InstanceType`	MARKOVITZ members halt the process (Prop. 4).
`SExpression.evaluate`	Inverted guard on `log` and `log10`.
`EvolutionaryStrategy`	Accepts the offspring on a *smaller* evaluation regardless of the declared objective.

Remark 18 (Reachability of the preserved defects from the generation path)**.** Not all entries in Table 5 are live. A static reachability check over the call graph rooted at `KP.generate` shows that `Knapsack.copy` is reached only through `KP.getKnapsack`, which no production call site invokes (it appears only in the verification harnesses), and that `Knapsack.getProductOfProfit` has no

call site at all. The overflow-prone product accumulator `pProfit` is therefore maintained but never read. These two are latent rather than active defects, and any hypothesis built on them must first establish an execution path that reaches them.

9 Catalogue of divergences

The table below separates three categories that are easy to conflate: a *contract violation* (the interface promises something the implementation does not deliver), a *latent defect* (incorrect code on an unreachable path), and a *documentation divergence* (the artefact behaves differently from how the surrounding literature describes it).

Table 6: Divergences between interface, implementation and description.

Item	Category	Statement
`crossoverRate`	Contract violation	Accepted, clamped, forwarded, never read. Crossover probability is 1. Prop. 5.
`maxWeight`, `maxProfit`	Contract violation	Enter the system only through $\lceil \log_2 \cdot \rceil$. Declared bounds are exceeded by up to a factor < 2; the nominal grid collapses onto $\lceil \log_2 N \rceil^2$ conditions. Cor. 2, Prop. 3.
MARKOVITZ types	Contract violation	Declared in the enumeration, absent from the dispatch, terminate the process. Effective type space is 16. Prop. 4.
Odd population size	Contract violation	Silently runs with $\mu+1$ individuals and overshoots the evaluation budget. Prop. 6.
`Knapsack.copy`	Latent defect	Double-decrements capacity. Unreachable from `KP.generate`.
`pProfit`	Latent defect	Product accumulator maintained on every pack, never read, overflow-prone.
Heuristic complexity	Documentation	Implemented as $\Theta(n^2)$ rescan, not $O(n \log n)$ sorted pass. Outputs agree; costs do not. Cor. 1, Prop. 1.
Capacity rule	Documentation	c is a fixed input parameter of `generate` (default 20), not a function of the generated weights. Any description of the capacity as a fraction of total weight refers to a different configuration or a different version of the generator, and must be reconciled before use.
Evaluation budget	Documentation	$E_{\max} = 100{,}000$ is hard-coded inside `KP.generate` and is not exposed in the signature.
`copy()` side effect	Design constraint	Advances the source individual's stream, coupling leaf streams to selection frequency. Prop. 7.
Static encoding state	Design constraint	Blocks concurrent runs with distinct θ in one process. Prop. 8.

Remark 19 (On the capacity rule, flagged for reconciliation)**.** The last documentation diver-

gence is the one most likely to affect downstream conclusions and is the least settled. The code inspected here takes c as an independent argument with default 20, while weights reach 32. Any methodological text stating that capacity is set to half the total item weight is describing either a different release of the generator or a driver program that is not part of the sources examined here. This should be resolved against the specific artefact used to produce any published dataset before the two descriptions are treated as interchangeable.

10 Summary of the formal model

The generator is the following stochastic optimisation procedure. Fix $\theta = (n, c, W_{\max}, P_{\max})$ and a type τ among the 16 implemented ones. Let $b_w = \lceil \log_2 W_{\max} \rceil$, $b_p = \lceil \log_2 P_{\max} \rceil$, $\ell = b_w + b_p$, $L = n\ell$. Then:

1. The search space is $\Omega = \{0,1\}^L$ and the decoding Γ_θ is a bijection onto $(\{1, \dots, 2^{b_w}\} \times \{1, \dots, 2^{b_p}\})^n$ (Prop. 2). This set, and no larger set, is what the generator covers.
2. The objective is $f_\tau : \Omega \to \mathbb{R}$ of Definition 5, evaluated by four destructive $\Theta(n^2)$ constructive solves per call.
3. The optimiser is a generational genetic algorithm with tournament selection of size k, unconditional one-point crossover, independent bit-flip mutation at rate p_m, and an external elite, run for $G = \lceil (E_{\max} - \mu)/\mu' \rceil$ generations under a hard-coded $E_{\max} = 100{,}000$ (Prop. 6).
4. The emitted instance is $\Gamma_\theta(\arg\max)$ over the elite trajectory, serialised with its *nominal* parameters in the file name.

Everything the exposed interface allows a user to vary reduces, in its effect on the reachable instance set, to the triple $(n, c, (b_w, b_p))$, while the τ parameter controls only where in that set the search concentrates. This is the precise sense in which the effective configuration space is smaller than the nominal one, and it is the starting point for any empirical study of the coverage of the generator.